OpenCL ядро содержит ошибки.

```
#define CE_WG_SIZE 256

//
================================================================================
// 1. ПРЯМОЙ ПРОХОД: Fused Sparse Softmax + CrossEntropy со сдвигом [:, :-1] vs [:, 1:]
// Сетка: Global = [CE_WG_SIZE, (seq_len - 1), batch_size], Local =
//
================================================================================
kernel void sparse_cross_entropy_forward(
    global const float* logits,       // [Batch, SeqLen, VocabSize]
    global const int* targets,        // [Batch, SeqLen]
    global float* token_losses,       // [Batch * (SeqLen - 1)]
    const int seq_len,
    const int vocab_size
) {
    int lid = get_local_id(0);
    int t   = get_group_id(1);        // [0 .. seq_len - 2]
    int b   = get_group_id(2);        // [0 .. batch_size - 1]

    int target_seq_len = seq_len - 1;
    if (t >= target_seq_len) return;

    __local float shared_buf[CE_WG_SIZE];

    // Читаем истинный токен со сдвигом (t + 1)
    int target_id = targets[b * seq_len + (t + 1)];
    global const float* row_logits = logits + (b * seq_len + t) * vocab_size;

    // Шаг 1: Параллельный поиск максимума по словарю (численная стабильность Softmax)
    float local_max = -INFINITY;
    for (int v = lid; v < vocab_size; v += CE_WG_SIZE) {
        local_max = fmax(local_max, row_logits[v]);
    }
    shared_buf[lid] = local_max;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int stride = CE_WG_SIZE / 2; stride > 0; stride >>= 1) {
        if (lid < stride) {
            shared_buf[lid] = fmax(shared_buf[lid], shared_buf[lid + stride]);
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    float max_val = shared_buf;
    barrier(CLK_LOCAL_MEM_FENCE);

    // Шаг 2: Параллельная сумма экспонент exp(logit - max_val)
    float local_sum = 0.0f;
    for (int v = lid; v < vocab_size; v += CE_WG_SIZE) {
        local_sum += native_exp(row_logits[v] - max_val);
```

```
    }
    shared_buf[lid] = local_sum;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int stride = CE_WG_SIZE / 2; stride > 0; stride >>= 1) {
        if (lid < stride) {
            shared_buf[lid] += shared_buf[lid + stride];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    // Шаг 3: Поток 0 записывает итоговую ошибку для токена (b, t)
    if (lid == 0) {
        float sum_exp = shared_buf;
        float target_logit = (target_id >= 0 && target_id < vocab_size) ? row_logits[target_id] :
max_val;
        float loss = log(sum_exp) - (target_logit - max_val);
        token_losses[b * target_seq_len + t] = loss;
    }
}

//
================================================================================
=====
// 2. РЕДУКЦИЯ СКАЛЯРНОГО LOSS (Усреднение token_losses в 1 число)
// Сетка: Global =, Local =
//
================================================================================
=====
kernel void reduce_mean_loss(
    global const float* token_losses, // [num_tokens]
    global float* out_loss,           // [1]
    const int num_tokens
) {
    int lid = get_local_id(0);
    __local float shared_sum[CE_WG_SIZE];

    float local_sum = 0.0f;
    for (int i = lid; i < num_tokens; i += CE_WG_SIZE) {
        local_sum += token_losses[i];
    }
    shared_sum[lid] = local_sum;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int stride = CE_WG_SIZE / 2; stride > 0; stride >>= 1) {
        if (lid < stride) {
            shared_sum[lid] += shared_sum[lid + stride];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    if (lid == 0) {
```

```
        out_loss = shared_sum / (float)num_tokens;
    }
}

//
================================================================================
=====
// 3. ОБРАТНЫЙ ПРОХОД: Сразу выдает grad_logits [Batch, SeqLen, VocabSize]
// Заменяет SliceOp + PadSlicedGradient + CrossEntropyBackward!
// Сетка: Global = [CE_WG_SIZE, seq_len, batch_size], Local =
//
================================================================================
=====
kernel void sparse_cross_entropy_backward(
    global const float* logits,       // [Batch, SeqLen, VocabSize]
    global const int* targets,        // [Batch, SeqLen]
    global const float* grad_loss,    // [1] (входящий градиент от loss, обычно 1.0f)
    global float* grad_logits,        // [Batch, SeqLen, VocabSize]
    const int batch_size,
    const int seq_len,
    const int vocab_size
) {
    int lid = get_local_id(0);
    int t   = get_group_id(1);        // [0 .. seq_len - 1]
    int b   = get_group_id(2);        // [0 .. batch_size - 1]

    int target_seq_len = seq_len - 1;
    global float* row_grad = grad_logits + (b * seq_len + t) * vocab_size;

    // Для последнего шага времени (t == seq_len - 1) градиент строго равен 0.0f!
    if (t >= target_seq_len) {
        for (int v = lid; v < vocab_size; v += CE_WG_SIZE) {
            row_grad[v] = 0.0f;
        }
        return;
    }

    __local float shared_buf[CE_WG_SIZE];

    int target_id = targets[b * seq_len + (t + 1)];
    global const float* row_logits = logits + (b * seq_len + t) * vocab_size;

    // 1. Поиск максимума по словарю
    float local_max = -INFINITY;
    for (int v = lid; v < vocab_size; v += CE_WG_SIZE) {
        local_max = fmax(local_max, row_logits[v]);
    }
    shared_buf[lid] = local_max;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int stride = CE_WG_SIZE / 2; stride > 0; stride >>= 1) {
        if (lid < stride) {
```

```
            shared_buf[lid] = fmax(shared_buf[lid], shared_buf[lid + stride]);
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    float max_val = shared_buf;
    barrier(CLK_LOCAL_MEM_FENCE);

    // 2. Сумма экспонент
    float local_sum = 0.0f;
    for (int v = lid; v < vocab_size; v += CE_WG_SIZE) {
        local_sum += native_exp(row_logits[v] - max_val);
    }
    shared_buf[lid] = local_sum;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int stride = CE_WG_SIZE / 2; stride > 0; stride >>= 1) {
        if (lid < stride) {
            shared_buf[lid] += shared_buf[lid + stride];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    float inv_sum_exp = 1.0f / shared_buf;
    float scale = grad_loss / (float)(batch_size * target_seq_len);

    // 3. Бесветвящийся расчет (softmax - one_hot) * scale
    for (int v = lid; v < vocab_size; v += CE_WG_SIZE) {
        float prob = native_exp(row_logits[v] - max_val) * inv_sum_exp;
        float indicator = select(0.0f, 1.0f, v == target_id);
        row_grad[v] = (prob - indicator) * scale;
    }
}
```

```
<kernel>:41:11: error: initializing 'float' with an expression
of incompatible type '__local float [256]'
    float max_val = shared_buf;
          ^         ~~~~~~~~~~
<kernel>:61:15: error: initializing 'float' with an expression
of incompatible type '__local float [256]'
        float sum_exp = shared_buf;
              ^         ~~~~~~~~~~
<kernel>:95:31: error: invalid operands to binary expression
('__local float [256]' and 'float')
        out_loss = shared_sum / (float)num_tokens;
                   ~~~~~~~~~~ ^ ~~~~~~~~~~~~~~~~~
<kernel>:147:11: error: initializing 'float' with an expression
of incompatible type '__local float [256]'
    float max_val = shared_buf;
          ^         ~~~~~~~~~~
<kernel>:165:30: error: invalid operands to binary expression
('float' and '__local float [256]')
    float inv_sum_exp = 1.0f / shared_buf;
                        ~~~~ ^ ~~~~~~~~~~
<kernel>:166:29: error: invalid operands to binary expression
('const __global float *' and 'float')
    float scale = grad_loss / (float)(batch_size *
target_seq_len);
                  ~~~~~~~~~ ^
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```